# Foundations of Spatial AI

Lecture 12: Gaussian-Based Active Planning

Based on lectures by **Valerio Spagnoli**
Sapienza University of Rome (`rvp-group.github.io/sai-teaching`)

November 2, 2026

**Abstract**

These lecture notes provide a rigorous and self-contained mathematical and algorithmic treatment of **Lecture 12: Gaussian-Based Active Planning** from the *Foundations of Spatial AI (2026/27)* course. We trace the transition from passive scene reconstruction to active perception and Next-Best-Pose planning, examine how **3D Gaussian Splatting (3DGS)** acts as a fast differentiable rendering engine for scoring candidate camera trajectories, analyze coverage-driven planning with **MAGICIAN**, and study a complete online semantic object-mapping architecture combining a 5 cm voxel planning state with multi-channel Gaussian signal rendering and horizon-5 beam search.

## Contents

**1 Active Perception and Active Mapping *(slides 1–4)*** **2**
- 1.1 What Is Active Perception? *(slides 1–2)* . . . 2
- 1.2 Active Mapping *(slide 3)* . . . 2
- 1.3 Our Task: Focus on the Objects *(slide 4)* . . . 3

**2 The Tool: 3D Gaussian Splatting *(slides 5–8)*** **4**
- 2.1 3D Gaussian Splatting Representation *(slide 5)* . . . 4
- 2.2 How Does 3DGS Work? *(slide 6)* . . . 5
- 2.3 Training Views, Novel Views, and What a Planner Does with 3DGS *(slides 7–8)* . . . 5

**3 MAGICIAN: Planning with Imagined Gaussians *(slides 9–11)*** **6**
- 3.1 MAGICIAN Architecture and Imagined Gaussians *(slides 9–10)* . . . 6
- 3.2 From Coverage to Objects *(slide 11)* . . . 7

**4 The Dual-Map System and Semantic Fusion *(slides 12–17)*** **8**
- 4.1 The System and Full Pipeline *(slides 12–13)* . . . 8
- 4.2 Semantics on a Real Room: From Pixels to Gaussians *(slides 14–15)* . . . 9
- 4.3 The Planning State: 5 cm Voxel Grid *(slides 16–17)* . . . 10

**5 Multi-Signal Planning and Beam Search *(slides 18–21)*** **10**
- 5.1 Three Signals, Three Questions *(slides 18–19)* . . . 10
- 5.2 Looking Ahead: Beam Search and Candidate Rendering *(slides 20–21)* . . . 12

**6 Experimental Results and Generalization *(slides 22–25)*** **13**
- 6.1 A Real Flight: Quantitative Progression and Object Lifecycle *(slides 22–24)* . . . 13
- 6.2 Same System, Another Task: Object Navigation *(slide 25)* . . . 14

**7 Summary and Key Takeaways *(slide 26)*** **14**
- 7.1 Comparison of Paradigms and Task Configurations . . . 14
- 7.2 Key Takeaways and Open Directions *(slide 26)* . . . 16

# 1 Active Perception and Active Mapping *(slides 1–4)*

## 1.1 What Is Active Perception? *(slides 1–2)*

Classical computer vision and spatial mapping pipelines often operate in a **passive** regime: a fixed sequence of sensor measurements is handed to the algorithm, which processes whatever arrives. In robotics and embodied AI, however, an agent controls its own motion and camera pose. This leads to the paradigm of **Active Perception**:

> *"What we perceive depends on what we decide to do."*

**Definition 1.1** (Passive vs. Active Perception)**.** Let $\mathcal{X}$ denote the space of sensor observations (e.g., RGB-D frames), $\mathcal{T} \subset \mathrm{SE}(3)$ the space of feasible camera poses, and $\mathcal{M}$ the space of internal scene representations (maps).

- **Passive Perception:** Given an exogenous sequence of observations $\{(I_t, D_t, T_t)\}_{t=1}^{N}$, the estimation algorithm computes a map $M_N = \Phi(\{(I_t, D_t, T_t)\}_{t=1}^{N})$ without influencing the poses $T_t$.
- **Active Perception:** At each discrete step $t$, the agent maintains a map $M_t \in \mathcal{M}$ updated from past observations and applies a decision policy $\pi : \mathcal{M} \times \mathcal{T}_{\text{task}} \to \mathcal{T}$ that selects the next viewpoint $T_{t+1} = \pi(M_t, \tau)$ conditioned on the task $\tau$, physically moves to $T_{t+1}$, and acquires $(I_{t+1}, D_{t+1})$.

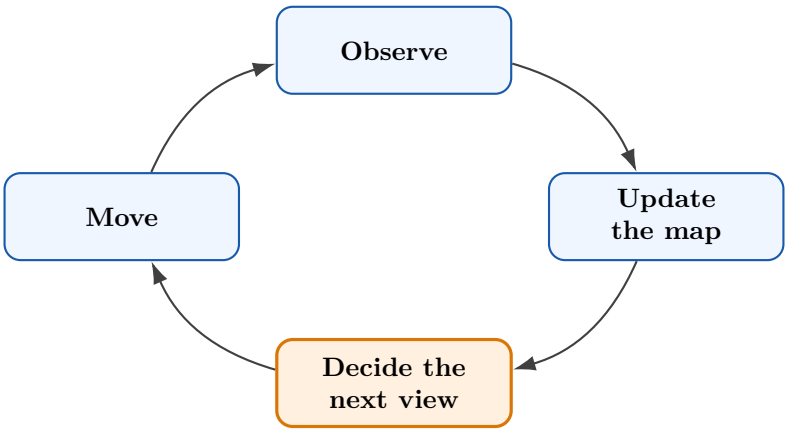



Figure 1: The closed-loop Active Perception cycle (redrawn from Slide 2). The highlighted orange block (*Decide the next view*) closes the loop between mapping and physical camera motion.

**Intuition — Why Passive Data Collection Fails Under a Budget**

Imagine walking into a cluttered living room with a camera and only being allowed to take $N = 50$ photographs. A passive agent moving along a pre-programmed path might spend 30 frames staring at a blank white wall or the floor, leaving complex furniture occluded or blurry. An active agent inspects its current partial map after every frame and asks: *"Where is my map still incomplete or uncertain, and which camera movement will fix that fastest?"*

## 1.2 Active Mapping *(slide 3)*

When the specific task $\tau$ is to reconstruct a complete 3D scene within a strict budget of views $B$, active perception specializes to **Active Mapping** (also known as the **Next-Best-View** or **Next-Best-Pose** problem).

At each discrete time step $t \in \{1, 2, \ldots, B\}$, the system receives an input tuple consisting of:

1. **Color frame** $I_t \in \mathbb{R}^{H \times W \times 3}$ (RGB image),
2. **Depth frame** $D_t \in \mathbb{R}_+^{H \times W}$ (metric depth per pixel),
3. **Camera pose** $T_t \in \mathrm{SE}(3)$ (the 6-DoF position and orientation in world coordinates).

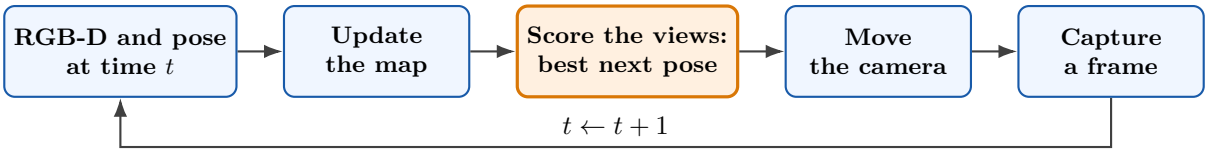



Figure 2: The sequential Active Mapping loop (redrawn from Slide 3). At each step $t$, the map is updated with $(I_t, D_t, T_t)$, candidate viewpoints are scored, and the camera moves to capture frame $t+1$.

**[ IMAGE PLACEHOLDER — Insert from Slide 3 (top right) ]**

*Visual Description:* Photorealistic 3D reconstruction of an indoor living room scene (white sofas, wooden coffee table, armchairs, floor lamp, window blinds) alongside stacked historical input frames (Color, Depth, Pose $T_t$).

**Suggested Caption:** Figure: Input stream of RGB, Depth, and camera poses $T_t$ (left) and the online reconstructed 3D scene map (right) from Slide 3.

### 1.3 Our Task: Focus on the Objects *(slide 4)*

Standard active mapping treats every square meter of surface area identically: a bare wall or carpet tile receives the same priority as an intricate object on a table. Slide 4 introduces the core thesis of this lecture:

> *"Not all surfaces matter the same: we care about the objects of the scene. To focus on objects, the robot must know what they are: we need semantics."*

**Definition 1.2** (Semantic Object-Centric Active Mapping)**.** Let $\mathcal{C} = \{1, \ldots, K\}$ be a set of semantic categories split into **background/structural classes** $\mathcal{C}_{\text{bg}}$ (e.g., wall, floor, ceiling) and **foreground object classes** $\mathcal{C}_{\text{obj}}$ (e.g., chair, cushion, vase, table, lamp). Given a view budget $B$, the objective of **Semantic Object-Centric Active Mapping** is to select a trajectory of poses $\tau = (T_1, \ldots, T_B) \in \mathcal{T}^B$ that maximizes the geometric completeness, spatial resolution (proximity), multi-angle coverage, and semantic classification accuracy across all object instances $o \in \mathcal{O}$ belonging to $\mathcal{C}_{\text{obj}}$.

To achieve this, the pipeline augments the raw RGB-D input with a 2D semantic segmenter (such as **OneFormer**) that predicts pixel-wise class labels, fuses those labels into the 3D representation, and directs the planner toward object regions requiring closer or novel viewing angles.

[ IMAGE PLACEHOLDER — Insert from Slide 4 ]

*Visual Description:* Four-panel visual summary showing: (1) Input: split RGB and depth image; (2) Classes from a segmenter (color-coded semantic segmentation mask); (3) Where the robot should look (3D grey mesh with bright orange highlighted target objects like pillows, coffee table items, side tables); (4) The map: split view showing photorealistic RGB and semantic class colors.

**Suggested Caption:** Figure: Overview of object-centric semantic active mapping (Slide 4), highlighting how semantic segmentation guides the planner to focus on scene objects (orange) rather than structural background.

# 2 The Tool: 3D Gaussian Splatting *(slides 5–8)*

## 2.1 3D Gaussian Splatting Representation *(slide 5)*

To both represent the scene at high visual fidelity and rapidly evaluate candidate viewpoints, the system builds upon **3D Gaussian Splatting (3DGS)** (Kerbl et al., ACM TOG 2023). Rather than using an implicit neural radiance field (NeRF) that requires costly volumetric ray-marching through a multilayer perceptron (MLP), 3DGS represents a scene explicitly as a collection of $M$ anisotropic 3D Gaussians (often on the order of 1.0 million Gaussians for a room mapped from 50 views).

**Definition 2.1** (Parameterized 3D Gaussian Primitive)**.** A 3D Gaussian primitive $g_k$ ($k \in \{1, \ldots, M\}$) is defined by the parameter tuple

$$g_k = (\boldsymbol{\mu}_k,\ \boldsymbol{\Sigma}_k,\ \alpha_k,\ \mathbf{c}_k,\ \mathbf{s}_k),$$

where:

- **Position (Mean):** $\boldsymbol{\mu}_k \in \mathbb{R}^3$ is the 3D center in world coordinates.
- **Shape (Covariance):** $\boldsymbol{\Sigma}_k \in \mathbb{S}^3_{++}$ is a $3 \times 3$ positive-definite covariance matrix defining the 3D ellipsoid's orientation and scale, parameterized via a rotation matrix $\mathbf{R}_k \in \mathrm{SO}(3)$ (stored as a unit quaternion) and a diagonal scaling matrix $\mathbf{S}_k = \mathrm{diag}(s_{k,x}, s_{k,y}, s_{k,z})$:
$$\boldsymbol{\Sigma}_k = \mathbf{R}_k \mathbf{S}_k \mathbf{S}_k^\top \mathbf{R}_k^\top.$$
- **Opacity:** $\alpha_k \in [0, 1]$ controls the radiative absorption/blending weight of the primitive.
- **Color:** $\mathbf{c}_k \in [0, 1]^3$ (or spherical harmonics coefficients) represents the RGB appearance.
- **Semantic / Utility Payload:** $\mathbf{s}_k$ stores task-specific attributes, such as semantic class vote slots or planning signal values.

The unnormalized 3D spatial density of Gaussian $k$ at point $\mathbf{x} \in \mathbb{R}^3$ is:

$$G_k(\mathbf{x}) = \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1}(\mathbf{x} - \boldsymbol{\mu}_k)\right).$$

[ IMAGE PLACEHOLDER — Insert from Slide 5 ]

*Visual Description:* Three-part figure of a reconstructed room from 50 views using 1.0 million Gaussians: (left) full 3D view of the room showing sofa, chairs, and table; (top right) close-up RGB render ('Color - 50 views'); (bottom right) close-up semantic render ('Classes - 50 views').

**Suggested Caption:** Figure: A living room represented by 1.0 million 3D Gaussians trained from 50 views, rendering both photorealistic color and dense semantic classes (Slide 5).

## 2.2 How Does 3DGS Work? *(slide 6)*

To render an image from a camera pose $T = (\mathbf{R}_c, \mathbf{t}_c) \in \mathrm{SE}(3)$, 3DGS projects each 3D Gaussian into the 2D image plane, sorts the Gaussians intersecting each pixel ray from front to back by depth, and alpha-composits their colors.

**Proposition 2.2** (Differentiable Splatting and Alpha-Compositing)**.** Given a viewing transformation $\mathbf{W}$ and the Jacobian $\mathbf{J}$ of the projective transformation at $\boldsymbol{\mu}_k$, the 3D covariance $\boldsymbol{\Sigma}_k$ projects to a 2D covariance matrix in pixel coordinates:

$$\boldsymbol{\Sigma}'_k = \mathbf{J}\mathbf{W}\boldsymbol{\Sigma}_k\mathbf{W}^\top\mathbf{J}^\top.$$

For a pixel $u \in \Omega$, let $\{1, \dots, K_u\}$ index the Gaussians overlapping $u$ sorted in ascending order of camera depth $z_1 \le z_2 \le \cdots \le z_{K_u}$. The effective 2D opacity at pixel $u$ is $\tilde{\alpha}_k(u) = \alpha_k G'_k(u)$, where $G'_k(u)$ is the 2D Gaussian evaluated at $u$. The rendered color $\hat{\mathbf{c}}(u)$ is given by linear blending:

$$\hat{\mathbf{c}}(u) = \sum_{k=1}^{K_u} w_k(u)\,\mathbf{c}_k, \qquad \text{where} \qquad w_k(u) = \tilde{\alpha}_k(u)\prod_{j=1}^{k-1}(1-\tilde{\alpha}_j(u)). \tag{1}$$

Here $T_k(u) = \prod_{j=1}^{k-1}(1-\tilde{\alpha}_j(u))$ is the accumulated transmittance from the camera to Gaussian $k$. During online training, the Gaussian parameters are optimized via gradient descent to minimize the pixel-wise photometric $\ell_1$ loss against the sensor image $\mathbf{c}(u)$:

$$\mathcal{L} = \sum_{u\in\Omega} \big|\hat{\mathbf{c}}(u) - \mathbf{c}(u)\big|. \tag{2}$$

## 2.3 Training Views, Novel Views, and What a Planner Does with 3DGS *(slides 7–8)*

Once optimized on training views collected along the robot's trajectory, 3DGS supports **novel view synthesis** from arbitrary test viewpoints $T \in \mathrm{SE}(3)$ at hundreds of frames per second. Crucially, the exact same alpha-compositing equation (1) can render any modality attached to the Gaussians:

- **Rendered RGB:** replacing $\mathbf{c}_k$ with RGB color $\mathbf{c}_k \in [0,1]^3$,
- **Rendered Depth:** replacing $\mathbf{c}_k$ with the camera-frame $z$-depth $d_k(T)$ of Gaussian $k$,
- **Rendered Semantic Classes:** replacing $\mathbf{c}_k$ with the class label or class probability vector of Gaussian $k$.

Slide 8 poses the central insight linking 3DGS to active planning:

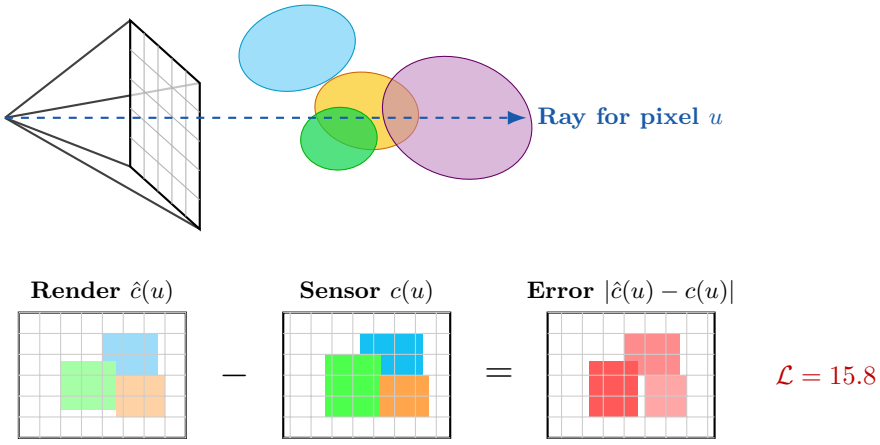



Figure 3: Illustration of 3D Gaussian Splatting (redrawn from Slide 6). Anisotropic 3D Gaussians project onto the sensor grid via $\hat{c}(u) = \sum_k w_k(u) c_k$. Comparing the rendered image against the ground-truth sensor image pixel-by-pixel yields the photometric error map $|\hat{c} - c|$ and scalar loss $\mathcal{L} = \sum_u |\hat{c}(u) - c(u)|$.

> *"3DGS is used for novel view synthesis, and rendering a new view is very fast. We use the renders to score a view for a task: $g(T, \tau) \in \mathbb{R}$."*

**Definition 2.3** (Render-Based View Scoring Function)**.** Let $s_k(\tau, T) \in \mathbb{R}_{\geq 0}$ denote a task-dependent scalar (or multi-channel vector) utility assigned to Gaussian $g_k$ under task $\tau$ when viewed from candidate pose $T \in \mathrm{SE}(3)$. The **Render-Based View Score** $g(T, \tau) \in \mathbb{R}$ is computed by rendering the utility signal across the image plane $\Omega$ and summing over all pixels:

$$g(T, \tau) = \sum_{u \in \Omega} \hat{s}(u; T, \tau) = \sum_{u \in \Omega} \sum_{k=1}^{K_u} w_k(u; T)\, s_k(\tau, T). \tag{3}$$

**Intuition — Why Rendering Solves Visibility and Occlusion for Free**

In traditional active mapping, scoring a candidate pose $T$ requires casting thousands of rays through a 3D voxel grid to check which surfaces are visible and which are occluded by walls or furniture in front of them. Because 3DGS already sorts Gaussians front-to-back and multiplies by the transmittance $T_k(u) = \prod_{j<k}(1 - \tilde{\alpha}_j(u))$, occluded Gaussians behind a solid wall automatically receive weight $w_k(u; T) \approx 0$! Thus, **replacing a Gaussian's color with a task utility turns the fast GPU rasterizer into an occlusion-aware information-gain calculator**.

# 3 MAGICIAN: Planning with Imagined Gaussians *(slides 9–11)*

## 3.1 MAGICIAN Architecture and Imagined Gaussians *(slides 9–10)*

Before introducing our semantic object-focused system, Slides 9–10 examine a recent state-of-the-art baseline for coverage-driven active mapping: **MAGICIAN** (*Li, S., et al., "MAGICIAN: Efficient long-term planning with imagined Gaussians for active mapping," CVPR 2026*).

A fundamental challenge in active mapping is that standard 3DGS only places Gaussians on surfaces that have *already been observed*. If the planner only renders existing Gaussians, a completely unexplored region appears empty and generates zero gain. MAGICIAN addresses this via three components:

1. **Imagined Gaussians:** A 3D **Volume Occupancy Network** takes the current reconstructed point cloud and RGB-D observations at time $t$ and predicts occupancy probabilities even in unobserved regions, instantiating *Imagined Gaussians* over predicted surfaces.

2. **Gain as a Novelty Render (Slide 10):** Each Imagined Gaussian is assigned:

$$\text{opacity } \alpha_k = \text{predicted occupancy} \in [0,1], \qquad \text{color } c_k = \text{novelty} \in \{0,1\},$$

   where novelty is 1 (visualized as bright red) if the surface has not yet been covered and 0 (grey) once seen. Rendering this novelty channel from a candidate pose $T$ and summing the pixel intensities directly computes the expected new surface coverage.

3. **Long-Term Planning via Beam Search:** Rather than greedily selecting only the immediate next view $T_{t+1}$, MAGICIAN evaluates multi-step candidate trajectories $\tau_1, \ldots, \tau_{N_b}$ of length $H$ using **beam search** to maximize cumulative coverage gain $G(\tau)$.

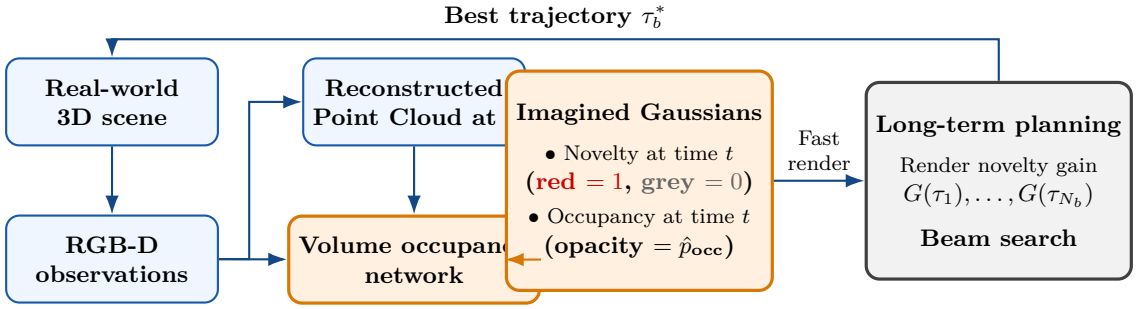



Figure 4: System architecture of MAGICIAN (redrawn from Slides 9–10). A volume occupancy network places Imagined Gaussians with opacity equal to predicted occupancy and color equal to novelty (1 = unseen, 0 = seen), enabling rapid evaluation of multi-step trajectories via beam search.

[ IMAGE PLACEHOLDER — Insert from Slide 10 ]

*Visual Description:* 3D cutaway view of a room populated with regular spherical/ellipsoidal Gaussians colored bright red (novelty = 1, unseen surfaces on walls/furniture) and grey (novelty = 0, already observed floor/furniture surfaces).

**Suggested Caption:** Figure: MAGICIAN's Imagined Gaussians in a room (Slide 10), where opacity encodes predicted occupancy and color encodes surface novelty (red = 1, grey = 0).

### 3.2 From Coverage to Objects *(slide 11)*

While MAGICIAN excels at rapid geometric exploration, Slide 11 highlights three critical limitations when the downstream robotics goal is object manipulation or semantic inspection:

1. **Single-glance saturation:** In pure coverage planning, once a surface is seen *once*—even from 6 m away at a grazing angle—its novelty drops to zero (*"Once seen, a surface counts zero, however far away"*). Yet a distant, single-view observation produces distorted, incomplete 3D reconstructions with severe holes (as shown by the coffee-table vase comparison on Slide 11).

2. **Lack of semantics:** *"A book and a wall weigh the same: no semantics."* Pure coverage wastes view budget mapping bare walls and floors at the same priority as tabletop objects.

3. **Robotics motivation:** *"Why: robots find, grasp and inspect objects; walls and floor are only where they move."*

> **[ IMAGE PLACEHOLDER — Insert from Slide 11 (top left) ]**
>
> *Visual Description:* Side-by-side close-up comparison of a decorative vase on a wooden coffee table: 'Seen once to mark as covered' (severely torn, missing geometry, black holes) vs. 'Seen multiple times to map' (clean, complete, high-resolution cylindrical vase reconstruction).
>
> **Suggested Caption:** Figure: Seeing an object once from afar marks it as 'covered' in MAGICIAN but yields poor geometry (left), whereas seeing it from multiple close viewpoints yields an accurate 3D map (right) (Slide 11).

# 4 The Dual-Map System and Semantic Fusion *(slides 12–17)*

## 4.1 The System and Full Pipeline *(slides 12–13)*

To map every object accurately along with its semantic class, our system maintains **two coupled representations** updated online from each posed RGB-D frame and OneFormer semantic segmentation mask:

1. **Online Gaussian Map (the final map):** Trained online from RGB-D frames and pixel classes; stores color, geometry, and semantic class votes. It serves both as the final high-resolution output map and as the fast differentiable renderer used by the planner.

2. **Voxel Map (the planning state):** Built from depth measurements alone at 5 cm resolution (with semantic class copied from the Gaussians inside each voxel). It tracks where the robot can safely fly (free vs. occupied vs. unknown space) and records detailed observation statistics per surface voxel: closest viewing distance, viewing directions, and frontier status.

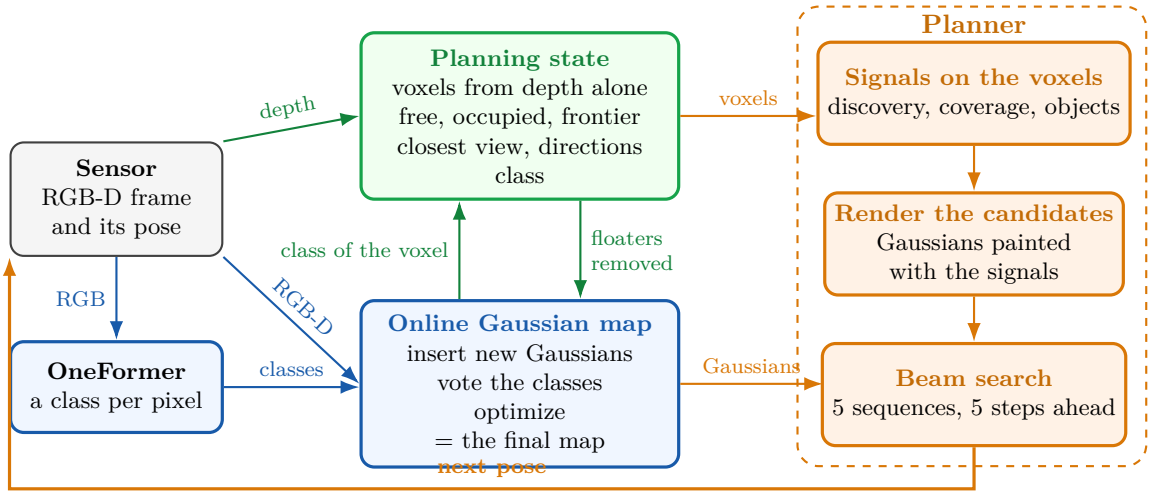



Figure 5: Complete architecture of the semantic Gaussian-based active mapping system (redrawn from Slides 13, 16, and 18). The Voxel Planning State and Online Gaussian Map exchange semantic classes and floater-pruning signals, while the Planner paints voxel utility signals onto the Gaussians to score 5-step beam search trajectories.

### 4.2 Semantics on a Real Room: From Pixels to Gaussians *(slides 14–15)*

2D semantic segmentation networks like OneFormer are noisy and view-dependent: the same chair might be labeled *armchair* in one frame, *sofa* from an oblique angle, or *cushion* when partially occluded. Storing a full probability distribution over $K \approx 150$ classes on $10^6$ Gaussians would consume excessive GPU memory. Slides 14–15 introduce a memory-efficient, depth-consistent **4-slot surface voting mechanism**.

**Algorithm 4.1** (4-Slot Depth-Gated Semantic Voting on Gaussians)**.** Each Gaussian $g_k$ maintains at most $L = 4$ semantic vote slots $\{(c_{k,\ell}, v_{k,\ell})\}_{\ell=1}^{4}$, where $c_{k,\ell} \in \mathcal{C} \cup \{\text{empty}\}$ is a class ID and $v_{k,\ell} \in \mathbb{R}_{\geq 0}$ is its accumulated vote weight. When frame $t$ arrives with depth map $D_t(u)$ and OneFormer segmentation mask $C_t(u)$:

1. **Visible Surface Gate (Slide 15):** For each pixel $u$ that Gaussian $g_k$ projects onto, let $z_k(T_t)$ be the distance of $\boldsymbol{\mu}_k$ along the camera ray and $D_t(u)$ be the sensor depth. Gaussian $g_k$ is allowed to receive a vote **if and only if** it lies on the visible physical surface:

$$\left|z_k(T_t) - D_t(u)\right| \leq \delta_{\text{surf}}. \tag{4}$$

   If $|z_k(T_t) - D_t(u)| > \delta_{\text{surf}}$ (for example, $g_k$ is occluded behind a foreground object or not on the visible surface, as in Frame 150 on Slide 14), it casts **no vote**.

2. **4-Slot Vote Accumulation:** If (4) holds with pixel class $c = C_t(u)$ and splatting weight $w_k(u)$:
   - If $c$ matches an existing slot $c_{k,\ell}$, increment $v_{k,\ell} \leftarrow v_{k,\ell} + w_k(u)$.
   - Else if an empty slot exists, initialize $(c_{k,\ell}, v_{k,\ell}) \leftarrow (c, w_k(u))$.
   - Else replace or decay the weakest slot if the incoming weight exceeds it.

3. **Dominant Label Assignment:** The semantic class of Gaussian $g_k$ is the slot with the highest accumulated vote:

$$\hat{c}_k = \arg \max_{\ell \in \{1,2,3,4\}} v_{k,\ell}.$$

**Intuition — Why the Depth Gate Prevents Semantic Bleeding Through Walls**

Suppose a Gaussian lies on a wall behind a brown armchair. When the camera looks at the armchair, the 2D projection of the wall Gaussian still falls inside the 2D pixel region labeled "armchair" by OneFormer. Without the depth check $|z_k(T_t) - D_t(u)| \leq \delta_{\text{surf}}$, the wall behind the chair would mistakenly accumulate "armchair" votes! Checking that the Gaussian's 3D depth matches the depth sensor's measurement ensures that **only Gaussians physically sitting on the visible surface receive semantic votes**.

[ IMAGE PLACEHOLDER — Insert from Slide 14 ]

*Visual Description:* 3D room map highlighting a single tracked Gaussian (circled in red/yellow near an armchair and window) at Frame 150, showing the 2D semantic frame, magnified frustum view, and the 4 semantic vote slots (all 0.00, 'not known yet = 1.00' because it is not on the visible surface).

**Suggested Caption:** Figure: Tracking a single Gaussian's 4 semantic vote slots at Frame 150 (Slide 14). Because the Gaussian is not on the visible surface in this frame, no vote is cast.

### 4.3 The Planning State: 5 cm Voxel Grid *(slides 16–17)*

While 3D Gaussians are ideal for rendering, path planning and collision checking require knowing explicitly which 3D volumes are free air, solid obstacles, or unmapped space. Furthermore, the Gaussian map can occasionally create "floater" artifacts in mid-air during optimization. Thus, Slides 16–17 define the **Planning State** on a regular 5 cm voxel grid constructed strictly from depth measurements.

**Definition 4.2** (Voxel Planning State)**.** Let $\mathcal{V}$ be a uniform 3D grid of cubic voxels of side length $\Delta v = 5\,\text{cm}$. Each voxel $v \in \mathcal{V}$ stores a state tuple:

$$\mathcal{S}(v) = \big(\text{state}(v),\ \text{frontier}(v),\ z_v,\ \mathbf{b}_{\text{seen}}(v),\ \mathbf{b}_{\text{feas}}(v),\ c(v)\big),$$

where:

1. **Occupancy State** $\text{state}(v) \in \{\texttt{free}, \texttt{occupied}, \texttt{unknown}\}$: updated via depth raycasting. Determines collision-free navigable space where the robot can fly, and prunes "floater" Gaussians that drift into `free` voxels.
2. **Frontier Flag** $\text{frontier}(v) \in \{0, 1\}$: marks `occupied` voxels that are immediately adjacent to `unknown` voxels, together with the outward normal toward the open/unknown side.
3. **Closest View Distance** $z_v \in \mathbb{R}_+ \cup \{\infty\}$: the minimum Euclidean distance from which voxel $v$ has been observed so far:
$$z_v^{(t)} = \min\big(z_v^{(t-1)},\ \|\mathbf{p}_{\text{cam}}(T_t) - \mathbf{p}_v\|_2\big).$$
4. **Directions Seen** $\mathbf{b}_{\text{seen}}(v) \in \{0, 1\}^{24}$: a 24-bit bitmask partitioning the viewing sphere around $v$ into 24 directional sectors (one bit per sector, set to 1 once $v$ is observed from that sector).
5. **Feasible Directions** $\mathbf{b}_{\text{feas}}(v) \in \{0, 1\}^{24}$: a 24-bit bitmask indicating which directional sectors around $v$ are unoccluded (line-of-sight through free space) and could still be used to view $v$.
6. **Semantic Class** $c(v) \in \mathcal{C}$: aggregated by majority vote from the dominant labels $\hat{c}_k$ of the 3D Gaussians whose centers $\boldsymbol{\mu}_k$ fall inside voxel $v$.

**[ IMAGE PLACEHOLDER — Insert from Slide 17 ]**

*Visual Description:* 3D voxelized reconstruction of 'room_0, pose 149: 5 cm voxels, colored by class', showing crisp 5 cm cubes colored according to semantic categories (green floor, blue window blinds, brown armchairs, purple rug).

**Suggested Caption:** Figure: The 5 cm Voxel Planning State at pose 149 in room_0, built from depth geometry and colored by the semantic class transferred from the interior Gaussians (Slide 17).

## 5 Multi-Signal Planning and Beam Search *(slides 18–21)*

### 5.1 Three Signals, Three Questions *(slides 18–19)*

With the voxel planning state tracking geometry, viewing history, and semantics, how should the planner value a candidate pose $T$? Slide 19 decomposes the active mapping objective into **three distinct signals** corresponding to three intuitive spatial questions.

**Definition 5.1** (Three Voxel Planning Signals)**.** For each occupied voxel $v \in \mathcal{V}$ and a candidate viewing pose $T \in \mathrm{SE}(3)$ at distance $d(v,T) = \|\mathbf{p}_{\mathrm{cam}}(T) - \mathbf{p}_v\|_2$ and viewing sector $m(v,T) \in \{1,\ldots,24\}$:

1. **Discovery Signal** $s_{\mathrm{disc}}(v,T)$ — *"Where does the known end?"*
   - **Reads:** frontier$(v)$ and its open side.
   - **Value:** Non-zero only on frontier voxels (frontier$(v) = 1$) when viewed from their open side; increases as the camera gets closer ($d(v,T)$ decreases), encouraging the robot to look across the boundary into unmapped space.
2. **Coverage Signal** $s_{\mathrm{cov}}(v,T)$ — *"Seen from one direction only?"*
   - **Reads:** $\mathbf{b}_{\mathrm{seen}}(v)$ (directions seen) and $\mathbf{b}_{\mathrm{feas}}(v)$ (feasible directions).
   - **Value:** Non-zero on every surface voxel seen from fewer than 2 directions ($\|\mathbf{b}_{\mathrm{seen}}(v)\|_1 < 2$), provided the candidate pose observes $v$ from a new, feasible direction ($b_{\mathrm{seen},m}(v) = 0$ and $b_{\mathrm{feas},m}(v) = 1$).
3. **Objects Signal** $s_{\mathrm{obj}}(v,T)$ — *"Which objects need a closer or new view?"*
   - **Reads:** semantic class $c(v)$, closest view distance $z_v$, and directions seen $\mathbf{b}_{\mathrm{seen}}(v)$.
   - **Value:** Non-zero on **open objects** ($c(v) \in \mathcal{C}_{\mathrm{obj}}$ that have not yet met retirement criteria). Rewards each **halving of the viewing distance** down to 1 m (i.e., $d(v,T) \le \frac{1}{2} z_v$ down to $\min z_v = 1\,\mathrm{m}$) as well as observations from **new directions** ($b_{\mathrm{seen},m}(v) = 0$). Once an object has been observed closely ($\le 1\,\mathrm{m}$) from all required feasible directions, it is marked **retired** and its signal becomes 0.

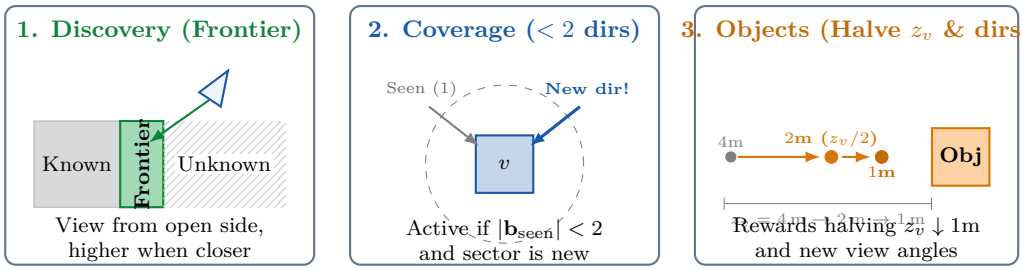



Figure 6: Geometric interpretation of the three voxel signals (Slide 19): (1) **Discovery** scores frontier voxels viewed from their open side; (2) **Coverage** scores surfaces seen from fewer than 2 directions when viewed from a new sector; (3) **Objects** scores foreground objects for each halving of viewing distance down to 1 m and for novel viewing sectors.

**Intuition — Channel-Modular Planning: Painting Signals onto Gaussians**

Notice the architectural elegance emphasized at the bottom of Slide 19: *"Each voxel gets a value per signal; each Gaussian takes the value of its voxel. The planner sees channels: a new signal needs no change to it."* Just as an RGB image has 3 color channels $(R,G,B)$, we can "paint" every Gaussian with a 3-channel utility vector $\mathbf{s}_k = (s_{\mathrm{disc}}, s_{\mathrm{cov}}, s_{\mathrm{obj}})$. A single rasterization pass from candidate pose $T$ produces a 3-channel utility image whose pixel sums tell the planner how much discovery, coverage, and object improvement pose $T$ delivers!

### 5.2 Looking Ahead: Beam Search and Candidate Rendering *(slides 20–21)*

A myopic (1-step greedy) planner can easily get trapped in a local corner if moving toward a distant object requires passing through a hallway that offers zero immediate reward on the first step. To plan ahead while keeping computation tractable, Slide 20 employs **Beam Search** over a discrete action graph of candidate camera moves.

**Algorithm 5.2** (Multi-Step Horizon Beam Search with Signal Rendering)**. Parameters:** Beam width $W = 5$ sequences, planning horizon $H = 5$ steps ahead, candidate neighbor action set $\mathcal{A}(T)$ (poses one move away in free voxel space).

1. **Initialization ($h = 0$):** Start with the singleton beam $\mathcal{B}_0 = \{(T_t)\}$ containing the current robot pose $T_t$.
2. **Iterative Expansion ($h = 1, \ldots, H$):**
   - Form the candidate set $\mathcal{C}_h = \{\tau \oplus T_{\text{next}} \mid \tau \in \mathcal{B}_{h-1},\ T_{\text{next}} \in \mathcal{A}(\text{last}(\tau))\}$.
   - For each sequence $\tau' = (T_{t+1}, \ldots, T_{t+h}) \in \mathcal{C}_h$, render the signal-painted Gaussian map along the sequence, discounting/zeroing out signals already collected by earlier poses in the same sequence $\tau'$ so that viewing the same surface twice in one trajectory is not double-counted:
   $$G(\tau') = \sum_{j=1}^{h} \Delta g(T_{t+j} \mid T_{t+1}, \ldots, T_{t+j-1}).$$
   - **Pruning:** Sort $\mathcal{C}_h$ by cumulative score $G(\tau')$ and retain only the top $W = 5$ highest-scoring sequences to form $\mathcal{B}_h$.
3. **Execution (Receding Horizon):** Select the highest-scoring sequence $\tau^* = \arg\max_{\tau \in \mathcal{B}_H} G(\tau)$, execute its first move $T^*_{t+1}$, capture a new RGB-D frame, update the maps, and repeat.

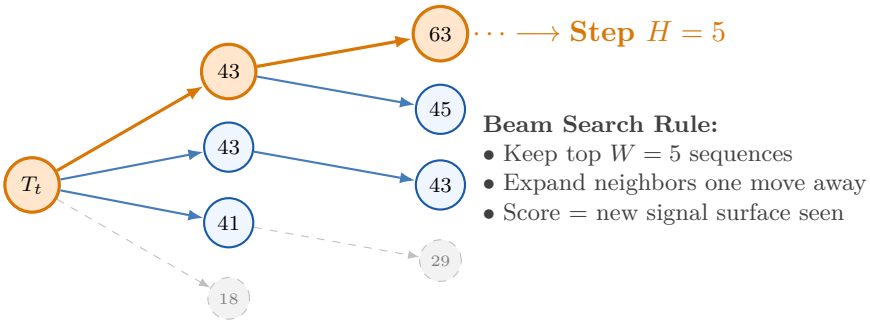



Figure 7: Conceptual illustration of Beam Search over pose sequences (adapted from Slide 20). At each depth, candidate neighbor poses are expanded and scored by rendering the signal-painted Gaussians; only the 5 best sequences (annotated with cumulative scores such as $43 \to 63$) are kept up to horizon $H = 5$.

[ IMAGE PLACEHOLDER — Insert from Slide 21 ]

*Visual Description:* 3D room view showing the current camera frustum (black wireframe) and a candidate next pose one move away (red/magenta wireframe) looking toward the coffee table and armchairs, accompanied by the multi-signal renders for Candidate 1.

**Suggested Caption:** Figure: Evaluating candidate poses one move away by rendering the Gaussian map once per signal channel (Slide 21).

# 6 Experimental Results and Generalization *(slides 22–25)*

## 6.1 A Real Flight: Quantitative Progression and Object Lifecycle *(slides 22–24)*

Slide 22 presents a full 600-pose autonomous flight in `room_0`, tracking both the normalized **work left** across the three signals (discovery, coverage, objects) and the lifecycle of semantic objects across three states:

- **Found:** Total number of distinct foreground objects discovered in the map so far.
- **Open:** Discovered objects that still require closer views (halving distance down to 1 m) or additional viewing directions.
- **Retired:** Objects whose distance and multi-angle viewing criteria have been completely satisfied ($s_{\text{obj}} = 0$).

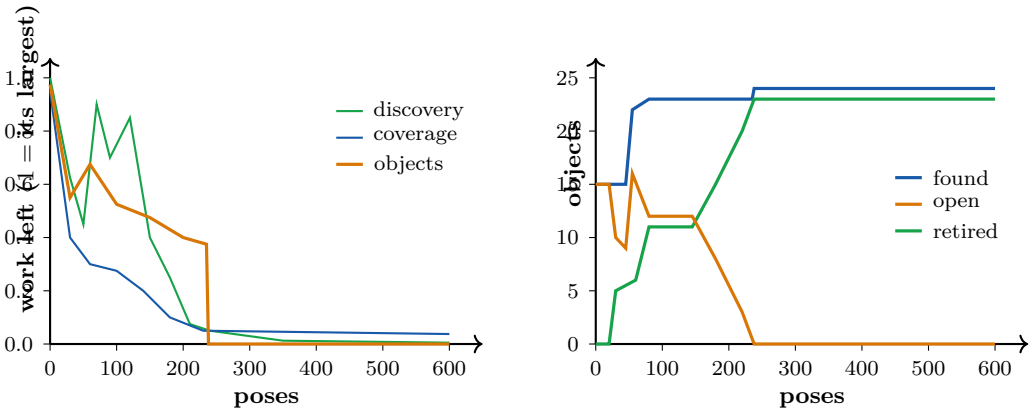



Figure 8: Quantitative dynamics during a 600-pose autonomous flight in `room_0` (redrawn from Slide 22). **Left:** Normalized work remaining for discovery, coverage, and objects drops sharply and finishes around pose 240. **Right:** Evolution of *found* (blue, reaching 24 objects), *open* (orange), and *retired* (green) objects; by pose 240, every discovered object has been thoroughly inspected and retired (`open` = 0).

**Intuition — Reading the Flight Curves on Slide 22**

Look closely at the two plots in Figure 8 around poses 50–120 and pose 240:

1. **Temporary spikes in "work left":** Around pose 55, the discovery and object curves jump *upward*. Why? Because the robot turns a corner and sees a new part of the room! New frontier voxels appear and the number of **found** objects jumps from 15 to 23, temporarily increasing the number of **open** objects waiting for close-up inspection.
2. **Task completion around pose 240:** By pose 240, the **retired** curve (green) catches up to the **found** curve (blue), driving **open** objects (orange) and "work left" to zero. This directly motivates one of the open research questions on Slide 26: *knowing automatically when to stop!*

[ IMAGE PLACEHOLDER — Insert from Slide 22 (top) and Slides 23–24 ]

*Visual Description:* Top of Slide 22: 3D trajectory (blue path) overlaid on RGB and semantic maps at pose 600 of 600. Slide 23: Full-bleed photorealistic online Gaussian render of room_0. Slide 24: 2x2 grid of online mapped scenes: room_2, office_0, office_2, office_3.

**Suggested Caption:** Figure: Online 3D Gaussian reconstructions and trajectories across Replica dataset rooms (`room_0`, `room_2`, `office_0`, `office_2`, `office_3`) from Slides 22–24.

### 6.2 Same System, Another Task: Object Navigation *(slide 25)*

A major strength of separating the **Planner/Renderer** from the **Signal Channels** is that changing the robot's task requires **zero changes** to the Gaussian map, differentiable renderer, or beam search planner—only the signals painted onto the Gaussians change. Slide 25 demonstrates this by contrasting **Object Mapping** with **Object Navigation** (*"Find a chair in a house never seen before"*):

- **Frontier Signal:** Rewards visible frontiers in line of sight to explore unknown rooms.
- **Relay Signal:** Guides the robot toward frontiers that are currently *out of sight* (e.g., through a doorway or corridor connecting rooms).
- **Target Signal:** A Vision-Language Model (VLM) detects bounding boxes of the target category (e.g., "chair") in each RGB frame; Gaussians falling inside the detected 2D boxes receive high target signal values.
- **Stopping Condition:** The robot stops automatically once it maneuvers close to a confirmed target object.

## 7 Summary and Key Takeaways *(slide 26)*

### 7.1 Comparison of Paradigms and Task Configurations

Table 1 synthesizes the active perception methods and task configurations covered across the lecture.

| Method / Task | Scene Representation | Signals Painted on Gaussians | Key Properties & Limitations |
|---|---|---|---|
| **Passive 3DGS** *(Kerbl et al., 2023)* | $M$ anisotropic 3D Gaussians $(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k, \alpha_k, \mathbf{c}_k)$ | RGB color $\mathbf{c}_k$ (for photometric loss $\mathcal{L}$) | Fast rendering; no viewpoint planning; suffers under limited view budgets. |
| **MAGICIAN** *(Li et al., CVPR 2026)* | Reconstructed Point Cloud + Volume Occupancy Net → **Imagined Gaussians** | Opacity $= \hat{p}_{\text{occ}}$, Color = Novelty $\in \{0, 1\}$ | Beam search over trajectories; maximizes raw surface coverage; ignores semantics & viewing distance (1 glance = 0 novelty). |
| **Semantic Object Mapping** *(This Lecture, Slides 12–24)* | Coupled **Online Gaussian Map** (4-slot voting) + **5 cm Voxel State** | 1. **Discovery** (frontiers)<br>2. **Coverage** ($< 2$ dirs)<br>3. **Objects** (halve $z_v \downarrow$ 1m, new dirs) | Depth-gated semantic voting; floater removal; maps every foreground object closely from multiple angles. |
| **Object Navigation** *(This Lecture, Slide 25)* | Same Online Gaussian Map + Voxel Planning State | 1. **Frontier** (in sight)<br>2. **Relay** (out-of-sight door)<br>3. **Target** (VLM 2D boxes) | Same renderer and 5-step beam search; stops when close to a confirmed target. |

Table 1: Comparison of passive 3DGS, coverage-driven MAGICIAN, Semantic Object Mapping, and Object Navigation.

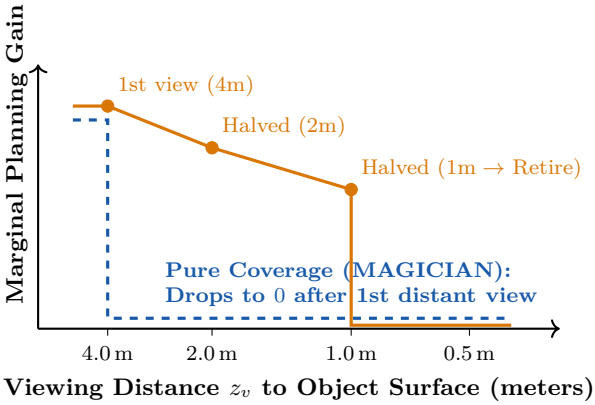



Figure 9: Qualitative comparison of marginal information gain as a function of viewing distance $z_v$ after an object is first spotted at 4.0 m. Pure coverage planners drop to zero gain immediately after the first distant observation, whereas the Object signal continues rewarding closer views (each distance halving down to 1.0 m) until the object is retired.

### 7.2 Key Takeaways and Open Directions *(slide 26)*

- **1. Render anything:** By replacing each Gaussian's RGB color $\mathbf{c}_k$ with a task utility signal $s_k$, candidate viewpoints are scored in a single fast, occlusion-aware rasterization pass: $g(T, \tau) = \sum_u \sum_k w_k(u) s_k$.
- **2. Plan on what you build:** The Online Gaussian Map and the 5 cm Voxel Planning State share information bidirectionally at every frame—Gaussians vote on depth-gated pixel semantics and label the voxels, while depth-raycasted voxels remove mid-air Gaussian floaters and compute spatial signals.
- **3. Value is a design choice:** Discovering unknown frontiers, covering single-sided surfaces, and getting closer to semantic objects (or navigating to a VLM-detected chair) are simply different signal channels fed into the exact same renderer and horizon-5 beam search planner.
- **4. Open problems & project ideas:** Key open challenges include learning **when to stop** automatically once scene work is exhausted, handling **dynamic/moving scenes**, operating under **unknown/noisy camera poses** (Active SLAM), deploying on a **real flying robot**, and designing new signal channels such as open-vocabulary **text queries**.

**Primary References Cited in the Lecture:**


[1] Kerbl, B., Kopanas, G., Leimkühler, T., & Drettakis, G. (2023). *3D Gaussian splatting for real-time radiance field rendering.* ACM Transactions on Graphics (TOG), 42(4), 1–14.

[2] Li, S., et al. (2026). *MAGICIAN: Efficient long-term planning with imagined Gaussians for active mapping.* In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR 2026).